ИТОГОВЫЙ ПРОЕКТ

Развёртывание сервиса анализа тональности в Kubernetes

Отчёт по дисциплине «Оркестрация и контейнеризация»

Виртуальная машина Ubuntu / VirtualBox

Выполнил: Абянов Дамир Шагидинович

Группа: _____

Проверил: _____

Дата: 10.11.2026

Москва, 2026

СОДЕРЖАНИЕ

1. Введение и архитектура стенда 3

2. Сборка и проверка REST API 4

3. Развёртывание в Kubernetes 5

4. Нагрузочный эксперимент 6

5. Наблюдаемость приложения 7

6. Бессерверный инференс и обучение с подкреплением 8

7. Оркестрация HPC и маршрутизация 9

8. Сравнение подходов, выводы и источники 10

1. Введение и архитектура стенда

Цель работы — развернуть сервис анализа тональности в Minikube, проверить маршрутизацию, балансировку, масштабирование и наблюдаемость. Исследовательская часть сопоставляет стенд с пятью работами по оркестрации инференса.

Ubuntu 24.04 в VirtualBox. Профиль sentiment-lab содержит два узла Ready. На ВМ использованы Minikube v1.39.0 и Kubernetes v1.37.0. Для стабилизации ВМ число vCPU увеличено с 2 до 4 при 6144 MiB памяти.

```
Client -> Ingress /api -> Service LoadBalancer -> Java pods
Metrics Server -> HPA -> Deployment -> Pod replicas
Actuator -> ServiceMonitor -> Prometheus -> Grafana
```

Компонент	**Роль**
Ingress	Маршрутизация /api для sentiment.local
Service	Единая точка входа и выбор готового пода
Deployment	Три базовые реплики Java с проверкой готовности
HPA	От 3 до 6 реплик при целевой утилизации CPU 60%
ServiceMonitor	Scrape /actuator/prometheus каждые 15 секунд



Рисунок 1 - Версии инструментов и ресурсы гостевой системы.

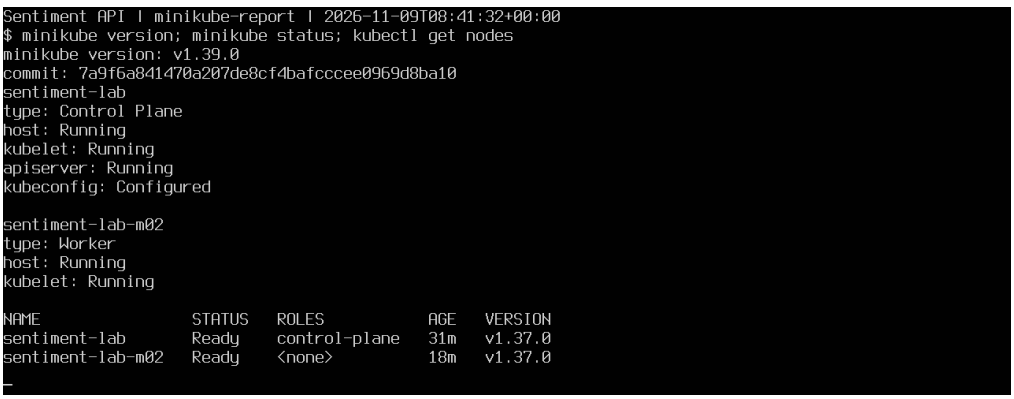


Рисунок 2 - Состояние Minikube и Kubernetes-узлов на виртуальной машине.

2. Сборка и проверка REST API

Spring Boot предоставляет REST API и Actuator. Демонстрационная модель токенизирует русский и английский текст и вычисляет разность числа слов из позитивного и негативного словарей. Нулевой результат соответствует классу neutral. Отрицания, сарказм и контекст модель не учитывает.

Проверки API. hello → positive, terrible → negative, table → neutral. Запрос без параметра text возвращает HTTP 400.

Образ собран на ВМ из готового JAR и runtime jlink. Docker показал 65 190 041 байт (65,2 МБ).

```
Sentiment API | minikube-report | 2026-11-09T08:07:10+00:00
$ cat Dockerfile.vm; docker image inspect sentiment-app:1.0.0
FROM maven:3.9-eclipse-temurin-21 AS jre
RUN jlink --add-modules java.se,jdk.crypto.ec,jdk.unsupported,jdk.management,jdk.zipfs \
    --strip-debug --no-header-files --no-man-pages --compress=zip-6 --output /rootfs/runtime \
    && mkdir -m 1777 /rootfs/tmp \
    && find /rootfs/runtime -type f -exec ldd {} + 2>/dev/null \
       | awk '{for(i=1;i<=NF;i++) if ($i ~ /^\/(usr\/)?lib/) print $i}' \
       | sort -u | xargs -r cp -L --parents -t /rootfs

FROM scratch
COPY --from=jre /rootfs/ /
COPY target/sentiment.jar /app/app.jar
USER 10001
EXPOSE 8080
ENTRYPOINT ["/runtime/bin/java", "-Xmx128m", "-XX:ActiveProcessorCount=2", "-XX:TieredStopAtLevel=1", "-ja
r", "/app/app.jar"]
Tags: [sentiment-app:1.0.0] | Size: 65190041 bytes
_
```

Рисунок 3 - Dockerfile и размер образа по docker image inspect.

```
Sentiment API | minikube-report | 2026-11-09T08:07:46+00:00
$ docker image inspect sentiment-app:1.0.0
65190041 bytes
$ curl http://127.0.0.1:18081/api/sentiment?text=hello
{"sentiment":"positive"}
$ curl http://127.0.0.1:18081/api/sentiment?text=terrible
{"sentiment":"negative"}
$ curl http://127.0.0.1:18081/api/sentiment?text=table
{"sentiment":"neutral"}
$ curl http://127.0.0.1:18081/actuator/health
{"status":"UP"}
$ curl http://127.0.0.1:18081/actuator/prometheus | grep process_cpu
process_cpu_time_ns_total 6.907E10
process_cpu_usage 0.21052631578947367
```

Рисунок 4 - Три класса тональности и метрики Actuator в контейнере на ВМ.

```
Sentiment API | minikube-report | 2026-11-09T09:42:43+00:00
$ curl -H "Host: sentiment.local" http://192.168.58.2:30373/api/sentiment?text=hello
hello      {"sentiment":"positive"}
terrible   {"sentiment":"negative"}
table      {"sentiment":"neutral"}
$ kubectl get endpointslice -l kubernetes.io/service-name=sentiment
NAME              ADDRESSTYPE   PORTS   ENDPOINTS                          AGE
sentiment-dwx4x   IPv4          8080    10.244.0.8,10.244.0.9,10.244.0.14   89m
_
```

Рисунок 5 - Ответы Kubernetes Service через NodePort.

3. Развёртывание в Kubernetes

Namespace sentiment отделяет проект от прежних лабораторных. Пробы /actuator/health управляют готовностью подов и перезапуском при сбое. Контейнер запускается от UID 10001.

```
bash scripts/build-image.sh
bash scripts/deploy.sh
kubectl --context sentiment-lab -n sentiment get deploy/sentiment
kubectl --context sentiment-lab -n sentiment get pods -l app=sentiment -o wide
```

Подтверждены 3 готовые реплики, LoadBalancer с адресом 10.105.153.124 и Ingress для sentiment.local. Три класса тональности проверены через NodePort сервиса.

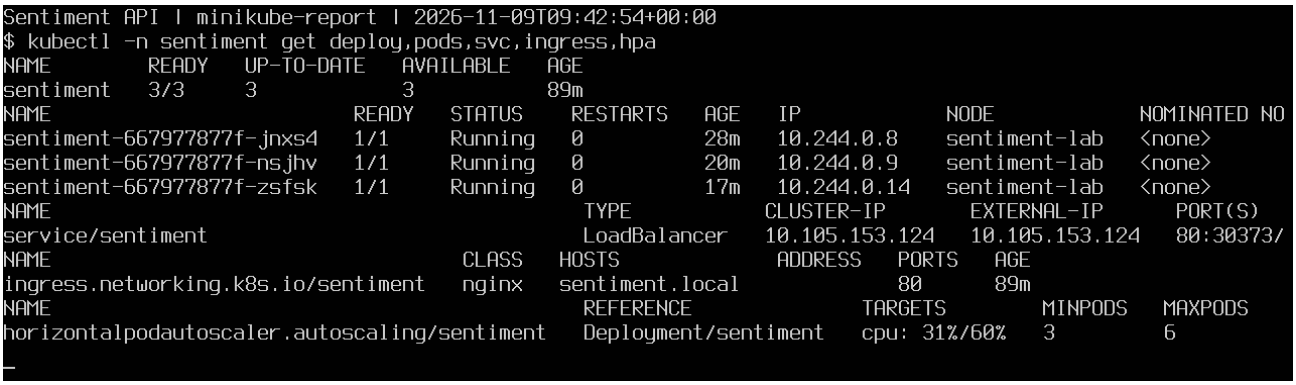


Рисунок 6 - Deployment, поды, Service, Ingress и HPA в namespace sentiment.

Параметры пода

Запрос CPU 50m, лимит 500m. Запрос памяти 128 MiB, лимит 256 MiB. Локальный тег образа 1.0.0 и imagePullPolicy: Never обеспечивают запуск из предварительно загруженного образа на каждом узле.

4. Нагрузочный эксперимент

Тест отправлял параллельные запросы через NodePort сервиса LoadBalancer. Заголовок X-Served-By идентифицирует под. Сравниваются количество готовых реплик, CPU и распределение успешных ответов между подами.

```
python3 scripts/load.py http://192.168.58.2:30373 180
kubectl -n sentiment get hpa -w
kubectl -n sentiment events --for hpa/sentiment
```

За 180 секунд получено 3022 успешных ответа и 79 ошибок. От трёх подов получено 1055, 971 и 996 ответов. При CPU 316% от запроса HPA увеличил желаемое число реплик с 3 до 6.

```
HPA timeline during the 180 s test | minikube-report
Source: kubectl get hpa, captured every 15 s during load

NAME        REFERENCE              TARGETS         MINPODS   MAXPODS   REPLICAS   AGE
sentiment   Deployment/sentiment   cpu: 316%/60%   3         6         3          90m
2026-11-09T09:44:15+00:00
NAME        REFERENCE              TARGETS         MINPODS   MAXPODS   REPLICAS   AGE
sentiment   Deployment/sentiment   cpu: 316%/60%   3         6         6          91m
2026-11-09T09:44:41+00:00

_
```

Рисунок 7 - Временной ряд HPA и событие масштабирования.

```
Sentiment API | minikube-report | 2026-11-09T09:46:12+00:00
$ python3 load.py http://192.168.58.2:30373 180
sentiment-667977877f-jnxs4: 1055
sentiment-667977877f-nsjhv: 971
sentiment-667977877f-zsfsk: 996
HTTP successes: 3022, errors: 79
_
```

Рисунок 8 - Успешные HTTP-ответы в разбивке по подам.

Интерпретация

Цель 60% относится к запросу CPU, поэтому для 50m порог составляет 30m в среднем на реплику. При отсутствии метрик HPA не может оценить текущую утилизацию. Число ответов по подам показывает фактическое распределение работы.

5. Наблюдаемость приложения

Actuator публикует счётчики и гистограммы HTTP. Prometheus получает их через ServiceMonitor. Grafana отображает CPU процесса Java, частоту запросов, p95 и число реплик со статусом сбора UP. Для HPA метрики CPU поступают из Metrics Server.

Prometheus собирает метрики приложения через ServiceMonitor. Дашборд Grafana подтверждён с данными CPU, частоты запросов, задержки p95 и числа целей UP.

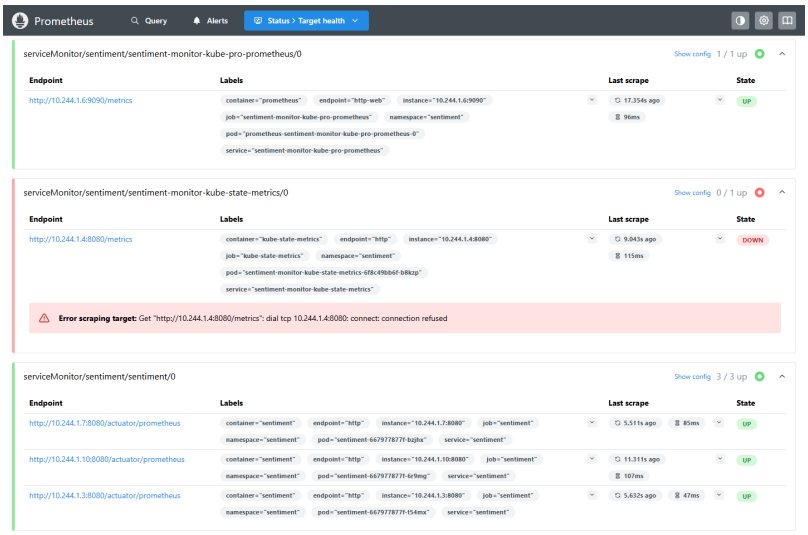


Рисунок 9 - Цели сбора метрик приложения в Prometheus.

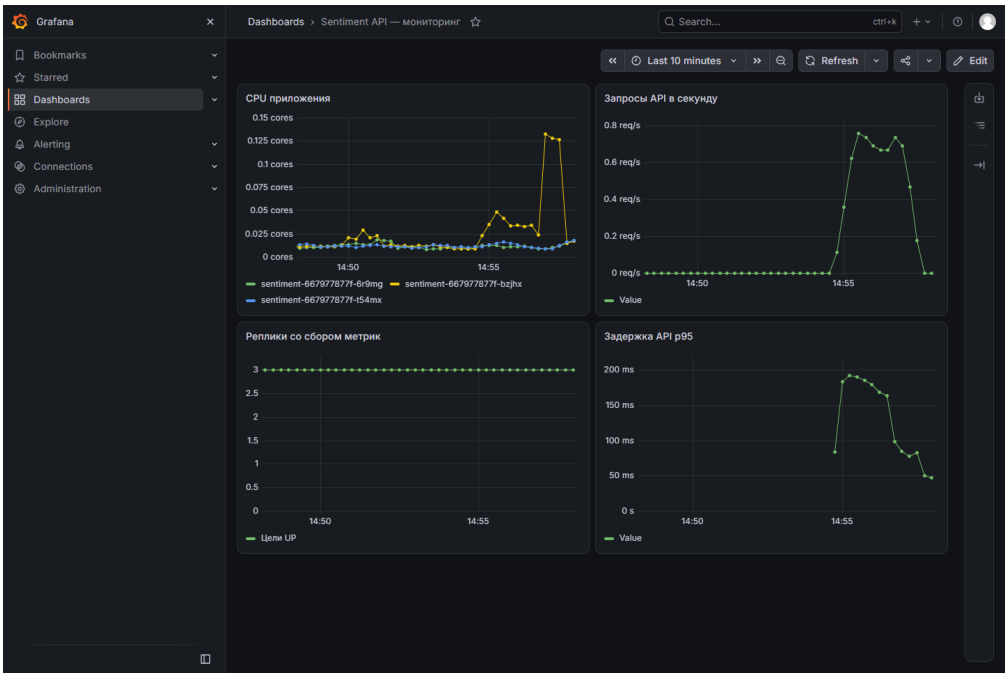


Рисунок 10 - Дашборд Sentiment API с данными из Prometheus.

6. Бессерверный инференс и обучение с подкреплением

Запрос «AI Kubernetes containerization 2024–2025» использован как отправная точка. Для сравнения выбраны пять работ, которые описывают разные уровни управления ресурсами и приводят экспериментальную оценку.

Remoe

Wentao Liu и др., 2025 · Бессерверный инференс MoE · arXiv:2512.18674

Метод. Редко вызываемые эксперты могут запускаться по требованию. Remoe прогнозирует активность экспертов по похожим запросам, резервирует память для основной модели и совместно подбирает размещение и число реплик.

Эксперимент. В Kubernetes-прототипе авторы сообщают о снижении стоимости до 57% и задержки холодного старта на 47% относительно выбранных базовых систем.

Применимость. Результат зависит от неравномерной активации экспертов и соотношения цен CPU и GPU. У небольшого словарного сервиса нет структуры MoE, поэтому декомпозиция только увеличила бы сетевые накладные расходы.

KIS-S

Guilin Zhang и др., 2025 · Масштабирование с обучением с подкреплением · arXiv:2507.07932

Метод. GPU-симулятор KISim моделирует инфраструктуру и инференс. Политика PPO в KIScaler обучается в симуляции, оптимизируя задержку и стоимость ресурсов, затем переносится без дообучения.

Эксперимент. На четырёх профилях трафика авторы получили прирост средней награды на 75,2% и снижение p95 до 6,7 раза по сравнению с CPU-базами.

Применимость. Сходство симулятора и стенда ограничивает переносимость. Для нашего приложения CPU HPA остаётся прозрачной базой сравнения. RL следует оценивать на повторяемых всплесках при одинаковом бюджете CPU и одинаковом SLO.

Общий вопрос

Обе работы распределяют ёмкость заранее, но используют разные сигналы. Remoe прогнозирует состав экспертов для конкретного запроса. KIS-S оценивает состояние контура масштабирования. При сравнении методов важно отдельно учитывать время готовности новой реплики и стоимость простаивающих ресурсов.

7. Оркестрация HPC и маршрутизация

AI Inference Scaling on HPC

Tim Trappen и др., 2025 · Интеграция Kubernetes, Slurm и vLLM · arXiv:2511.21413

Метод. На RAMSES Kubernetes обслуживает пользовательский контур, Slurm выделяет HPC-ресурсы, а vLLM исполняет запросы. Такая схема связывает интерактивный сервис с пакетным планировщиком.

Эксперимент. Авторы проверили 100, 500 и 1000 одновременных запросов. В предварительном тесте дополнительная задержка составила около 500 мс.

Применимость. Итог зависит от наличия свободных HPC-ресурсов. Если очередь Slurm длинная, время выделения ускорителя следует измерять отдельно от задержки уже запущенной реплики.

Cloud Native System for LLM Inference Serving

Minxian Xu и др., 2025 · Облачная архитектура инференса · arXiv:2507.18007

Метод. Работа рассматривает контейнеризацию, микросервисную декомпозицию и динамическое планирование в одном контуре. Эффект эластичности исследован на Kubernetes с автоматическим изменением числа экземпляров.

Эксперимент. Эксперименты показывают реакцию на колебания нагрузки. Оценка опирается на выбранную конфигурацию и рабочий профиль.

Применимость. Для Java API полезно разделить проверку готовности и контроль задержки. Реплика должна войти в балансировку после готовности, а p95 нужно измерять отдельно от CPU.

Block

Wei Da и Evangelia Kalyvianaki, 2025 · Прогнозная маршрутизация запросов · arXiv:2508.03611

Метод. Block оценивает ожидаемую нагрузку входящих запросов по их контексту и характеристикам оборудования. Распределённые планировщики используют прогноз, чтобы выбирать экземпляр и управлять ёмкостью.

Эксперимент. На стенде из 12 GPU авторы сообщают о росте обслуживаемой нагрузки до 16,7% и сокращении p99 до 49,5% относительно эвристических планировщиков.

Применимость. Эффект зависит от качества прогноза длины ответа. Для словарной модели время работы теснее связано с длиной входа. При неоднородных запросах можно сравнить маршрутизацию по длине входа с обычной балансировкой.

8. Сравнение подходов, выводы и источники

Препринт	Механизм	Что измерять на стенде
Remoe	Прогноз активации экспертов и эластичное размещение	Холодный старт и стоимость резервирования
KIS-S	Обучаемая политика управления GPU-репликами	p95, реплики и время реакции при всплеске
HPC Scaling	Связь Kubernetes, Slurm и vLLM	Время выделения ресурса отдельно от HTTP
Cloud Native System	Контейнеризация и динамическое масштабирование	Готовность реплик и пропускная способность
Block	Прогноз нагрузки каждого запроса	Распределение по подам и хвост задержек

Для сравнения подходов нужны повторные прогоны с одинаковым трафиком и ресурсами. Сопоставляются p95, доля ошибок, длительность превышения SLO, время до готовности реплики и реплико-секунды. Результаты статей относятся к разным стендам, моделям и нагрузкам.

Результаты

На ВМ подтверждены двухузловой кластер, образ 65,2 МБ, три готовых пода, распределение 3022 успешных ответов и масштабирование с 3 до 6. Архив содержит локальную историю Git.

Диагностика стенда

При загрузке ВМ наблюдались задержки планировщика и тайм-ауты API Kubernetes. Для изоляции прежних сервисов создан отдельный профиль. В Deployment добавлена startupProbe после того, как запуск Spring Boot в кластере занял около 78 секунд.

Материалы для воспроизведения

Архив проекта содержит исходный код, Dockerfile, Kubernetes-манифесты, Helm values, дашборд, README и локальный Git bundle. Дополнительные скриншоты кода, YAML и мониторинга находятся в docs/screenshots.

Препринты

Wentao Liu и др., 2025. Remoe. arXiv:2512.18674
Guilin Zhang и др., 2025. KIS-S. arXiv:2507.07932
Tim Trappen и др., 2025. AI Inference Scaling on HPC. arXiv:2511.21413
Minxian Xu и др., 2025. Cloud Native System for LLM Inference Serving. arXiv:2507.18007
Wei Da и Evangelia Kalyvianaki, 2025. Block. arXiv:2508.03611

Документация

Запуск Minikube · Horizontal Pod Autoscaling · kube-prometheus-stack